COMPANION GUIDE · NO PRIOR HARDWARE KNOWLEDGE ASSUMED

The Beginner's Guide to a Systolic Accelerator

What this chip does, why it is built the way it is, and what every single line of its code means — written for someone who has never seen a hardware description language before.

`13 modules` `~800 lines of SystemVerilog` `every line explained` `glossary included` `industry context`

HOW TO READ THIS DOCUMENT

Sections 1–5 are background: what a chip is, what the code is describing, and the maths the design performs. If you already know what a flip-flop and a clock edge are, skip to section 6, which walks through every file line by line. Section 10 collects every new term in one place — it is fine to jump there whenever a word is unfamiliar.

The companion document architecture_and_dataflow.html covers the same design at a technical level, with **animated cycle-by-cycle figures** of the delay pipes and the matrix multiply. Read this one first, then that one.

1 What is this thing?

This project is the **design and verification of a small piece of custom computer hardware**. It is not a program that runs on a computer. It is a description of a circuit — of actual wires, adders and memory cells — written in a language that a tool can turn into a real chip layout.

The circuit does one job: it multiplies matrices. Specifically, it computes **C = A × W**, where `W` is a fixed 4×4 grid of numbers and `A` is a stream of 4-number vectors fed in one after another.

> **WHY ANYONE WOULD BUILD THIS**
>
> Matrix multiplication is what neural networks spend almost all of their time doing. Every layer of a neural network is, at heart, "multiply this vector of inputs by that matrix of weights, then add them up". A general-purpose processor can do it, but slowly and while burning a lot of power. A circuit built *specifically* to do it can be dozens or hundreds of times more efficient — and that is exactly what a Google TPU, an Apple Neural Engine or an NVIDIA Tensor Core is. This project is a small, complete, honest version of the same idea.

An analogy: the factory line

Imagine a factory floor laid out as a 4×4 grid of workstations, 16 in total.

- **Each workstation is given one number to remember** — its *weight*. It is handed this number once at the start of the shift and keeps it all day. That is the "weight-stationary" part.
- **Parts enter from the left** and move right, one station per minute. Every station that a part passes through multiplies it by its remembered number.
- **A running total travels downward** through each column. When it reaches a station, that station adds its own multiplication result to the total and hands it on.
- **Finished totals fall off the bottom** of the grid. Each one is a completed dot product.

Nothing ever waits. Every station works every minute. The parts keep moving, the totals keep moving, and the weights never move at all. That constant, rhythmic pulse of data through a fixed grid is why it is called **systolic** — the word comes from *systole*, the contraction of a heart that pushes blood through the body.

What is actually in this repository

Folder	What is in it	Is it "the chip"?
`design_rtl/`	13 SystemVerilog files describing the circuit	**Yes** — this is the design
`test_benches/`	13 test programs that poke the design and check its answers	No — testing only, never becomes hardware

Folder	What is in it	Is it "the chip"?
`indi_ver_rep/`	Test results and raw simulator transcripts	No — evidence
`DOCS/`	Documentation, including this file	No
`context_mkdwn/`	Why decisions were made; derivations of the timing numbers	No

2 What a chip actually is

Before any code makes sense, four ideas.

2.1 Everything is wires and gates

A digital chip is millions of tiny switches (transistors) wired into **logic gates** — AND, OR, NOT — which combine into adders, multipliers and multiplexers. A wire carries either a 0 or a 1. A group of wires carries a number: 8 wires can carry any value from 0 to 255, or −128 to +127 if you agree to treat the top wire as a sign.

Logic built purely from gates is called **combinational**. It has no memory: change the inputs and the outputs change almost immediately (after a tiny propagation delay). `2 + 3 = 5` happens as fast as electricity can cross the adder.

2.2 The clock, and why it exists

A **clock** is a wire that alternates 0,1,0,1,... millions or billions of times per second. It is the heartbeat that everything in the chip marches to. In this design the clock is drawn with a period of 10 ns in simulation, so one "cycle" is 10 ns.

The moment the clock goes from 0 to 1 is the **positive edge** (or *posedge*). This is the instant at which the chip's memory elements take a snapshot.

2.3 Registers: the chip's short-term memory

A **register** (or **flip-flop**) is a one-bit storage cell. It has an input, an output and a clock. On every positive clock edge it copies whatever is on its input to its output, and holds that value until the next edge. A 32-bit register is just 32 of them side by side.

> **THE SINGLE MOST IMPORTANT CONSEQUENCE**
>
> A register introduces a **delay of exactly one clock cycle** between its input and its output. Almost every timing number in this entire design — 3, 6, 7, 8, 19 — is a count of how many registers a value has to pass through. Once you internalise "one register = one cycle", the whole design becomes arithmetic.

2.4 Pipelining: why chips are fast

Suppose a calculation takes four steps. You could do all four in one clock cycle, but then the cycle must be long enough for the slowest path — so the clock is slow.

Instead you put a register between each step. Now each cycle only has to be long enough for *one* step, so the clock can be four times faster. Any individual result takes four cycles to emerge (that is the **latency**), but a new result comes out *every* cycle once the pipeline is full (that is the **throughput**).

```
Without a pipeline           With a 4-stage pipeline
-------------------          -------------------------------
cycle 1: A B C D  -> r1      cycle 1: A₁
cycle 2: A B C D  -> r2      cycle 2: A₂ B₁
cycle 3: A B C D  -> r3      cycle 3: A₃ B₂ C₁
                             cycle 4: A₄ B₃ C₂ D₁  -> r1
slow clock, 1 result/cycle   cycle 5: A₅ B₄ C₃ D₂  -> r2
                             fast clock, still 1 result/cycle
                             ...but each cycle is 4× shorter
```

This accelerator is one long pipeline. A result takes 7 cycles to appear, but a new result appears every cycle. That trade — latency for throughput — is the defining move of digital design, and it is why "it takes 8 cycles" is not a complaint about the design.

3 Reading SystemVerilog: the 12 things you need

SystemVerilog is a **hardware description language**. It looks like C, and that is the single biggest trap for a newcomer: it does not *execute*, it *describes*. When you write a loop, you are not asking the chip to iterate — you are asking the tool to **build four copies of something**.

`module … endmodule`
A named block of hardware with a list of connections. The closest software analogy is a class, but a better one is a physical component with pins. Everything in this design is a module.
`input / output`
The pins. `input logic clk` means "a one-wire input called `clk`".
`logic`
The universal wire type. `logic [7:0] x` is an 8-wire bundle named `x`, with bit 7 on the left (most significant) and bit 0 on the right. The tool works out whether it needs to be a register or a plain wire from how you use it.
`signed`
Declares that the top bit means "negative". `logic signed [7:0]` holds −128 … +127. Leave it off and the same 8 wires mean 0 … 255. Getting this wrong is one of the classic silent bugs.
`parameter`
A compile-time constant that can be overridden when the module is instantiated. `parameter ROWS = 4` lets the same source describe a 4×4 or an 8×8 array. Parameters are how hardware is made reusable.

`assign`
A permanent, continuous connection — a piece of wire. `assign y = a + b;` means "there is an adder here, and its output is permanently connected to `y`". It is not an event that happens once; it is always true.
`always_ff @(posedge clk)`
"Build registers." Everything assigned inside this block becomes a flip-flop that updates on the rising clock edge. `ff` stands for *flip-flop*.
`always_comb`
"Build combinational logic." Everything assigned inside is pure gates with no memory, recomputed whenever any input changes.
`<= (non-blocking)`
Used inside `always_ff`. It means "all the right-hand sides are read **first**, using the values from before this clock edge, and only then are the left-hand sides updated." This is what makes `a <= b; b <= a;` a genuine swap rather than a bug, and it is what makes shift registers work.
`= (blocking)`
Used inside `always_comb`. Ordinary immediate assignment, like in C. Using `=` inside `always_ff` is a classic beginner error that produces a circuit which simulates differently from how it synthesises.
`generate / genvar`
A loop that runs at **compile time** and stamps out copies of hardware. `for (r = 0; r < 4; r++) processing_element pe(...)` does not run four times at runtime — it creates four separate physical PEs.
`[0:ROWS-1]`
An **unpacked array** — four separate bundles of wires, written after the name: `logic [7:0] a [0:3]` is four 8-bit buses. Compare `logic [31:0] a`, a **packed** 32-wire bundle you can slice. The difference matters: packed can be treated as one number, unpacked cannot.

Two syntax details that appear constantly in this code

Part-select with `+:`

```
a_rdata[r*DATA_W +: DATA_W]     // DATA_W bits starting at bit r*DATA_W
```

Read it as "start here, take this many bits, going up". With `DATA_W = 8` and `r = 2` it means bits `[23:16]`. It exists because `[r*8+7 : r*8]` is illegal when `r` is not a constant, and this form is legal.

Width casts and sign casts

```
32'(weight_q * a_in)            // force the result to be 32 bits wide
signed'(w_rdata[c*8 +: 8])      // reinterpret those 8 bits as a signed number
PSUM_W'(signed'( ... ))         // then sign-extend it out to 32 bits
```

Casts are everywhere in this design because SystemVerilog's automatic width and sign rules are notoriously surprising. Writing the cast makes the intent explicit and stops the tool from silently zero-extending a negative number.

4 The maths: matrix multiplication

A **matrix** is a rectangular grid of numbers. A **vector** is a single row or column of them.

The core operation is the **dot product**: take two lists of numbers of the same length, multiply them element by element, and add up all the products.

```
a = [ 1,  2,  3,  4 ]
b = [ 2,  1, -3,  0 ]

a · b  =  (1×2) + (2×1) + (3×−3) + (4×0)
       =    2   +   2   +   −9   +   0
       =  −5
```

Matrix multiplication is just a lot of dot products arranged neatly. To multiply activation vector `A` by weight matrix `W`, the result's element in column *c* is the dot product of `A` with column *c* of `W`:

```
              W  (4×4, stationary)
              c0   c1   c2   c3
       r0 [    2   −1    3    0 ]
       r1 [    1    4   −2    5 ]
       r2 [   −3    2    1    1 ]
       r3 [    0    1    2   −4 ]

A = [ 1, 2, 3, 4 ]        ← one activation vector, one element per ROW of W

C[0] = 1×2  + 2×1  + 3×(−3) + 4×0    =  −5     ← uses column 0 of W
C[1] = 1×(−1)+ 2×4  + 3×2    + 4×1   =  17     ← uses column 1
C[2] = 1×3  + 2×(−2)+ 3×1    + 4×2   =  10     ← uses column 2
C[3] = 1×0  + 2×5  + 3×1    + 4×(−4) =  −3     ← uses column 3

C = [ −5, 17, 10, −3 ]
```

Written as one formula, this is the entire specification of the chip:

$$C[i][c] = \Sigma_r \; A[i][r] \times W[r][c]$$

HOW THIS MAPS ONTO THE GRID

The 4×4 array has exactly 16 processing elements and `W` has exactly 16 numbers. **PE at row *r*, column *c* holds `W[r][c]`.** Element *r* of the activation vector enters row *r* from the west and passes through all four PEs in that row. Column *c*'s running total collects `A[r] × W[r][c]` from each row as it descends. When it exits the bottom of column *c*, it is `C[c]`.

Every multiply in the formula happens in exactly one place, and every sum happens on exactly one wire. The grid *is* the equation.

Why int8 in and int32 out

The inputs are 8-bit signed integers (**int8**): −128 to +127. Multiplying two of them gives at most `128 × 128 = 16 384`, which needs 15 bits. Adding four such products needs `4 × 16 384 = 65 536`, about 17 bits. The design uses 32 bits for the running total, which is far more than needed here — but it is the standard width, it costs little at this size, and it leaves headroom if the array ever grows.

Using 8-bit inputs at all is a deliberate industry-standard choice called **quantisation**: neural networks are usually trained in 32-bit floating point and then converted to 8-bit integers for deployment, because an 8-bit multiplier is roughly *16 times* smaller and cheaper than a 32-bit floating-point one, and the accuracy loss is usually negligible.

5 Why a systolic array? (and why not a CPU)

The real problem is memory, not maths

Multiplying two numbers is cheap. *Fetching* two numbers from memory is expensive — it costs far more energy and takes far longer than the multiply itself. A rough industry rule of thumb: reading a value from off-chip DRAM costs on the order of **hundreds of times** more energy than the arithmetic you then do with it.

So the whole game in accelerator design is **data reuse**: get a number on-chip once, then use it as many times as possible before throwing it away.

How a CPU does it, and why that is slow

```
for i in vectors:
    for c in columns:
        sum = 0
        for r in rows:
            sum += A[i][r] * W[r][c]      ← two memory reads per multiply
        C[i][c] = sum
```

A classic CPU executes this one multiply at a time, and for each multiply it must fetch the operands, decode an instruction, and update a loop counter. Caches help enormously, but the fundamental structure — fetch, decode, execute, write back, for every single operation — is overhead that the arithmetic does not need.

What the systolic array does instead

Weights are fetched once

Four SRAM reads load all 16 weights. Every activation vector afterwards reuses them with **zero** further weight traffic. Stream 200 vectors and you have amortised those four reads 200 times over.

Activations are fetched once

One 32-bit read supplies all four elements of a vector. Each element is then reused across all four columns as it travels east, without ever being read again.

Partial sums never touch memory

The running total lives on a wire between two adjacent PEs. In a CPU it would be a register write and read every step; here it is simply passed down.

No instructions at all

There is no fetch, no decode, no branch prediction. A tiny state machine says "we are in COMPUTE" and 16 multipliers do their thing. All the transistors are doing arithmetic.

INDUSTRY CONTEXT

This is precisely the architecture of Google's first **Tensor Processing Unit** (2016), which used a 256×256 weight-stationary systolic array of 8-bit multipliers — 65 536 PEs, versus 16 here. The structure is identical; only the numbers differ. The same family of ideas appears in NVIDIA's Tensor Cores, Apple's Neural Engine, and essentially every "NPU" shipped in a phone since 2018.

The three named dataflows you will meet in the literature are **weight-stationary** (weights stay put — this design), **output-stationary** (each PE owns one output and accumulates in place), and **row-stationary** (used by the MIT Eyeriss chip, which optimises for convolution). Weight-stationary is the simplest and wins when the same weights are reused across many inputs, which is the common case for a fully-connected neural network layer.

The cost of all this

It is worth being honest about what the architecture gives up:

- **It does exactly one thing.** A CPU can run a web browser. This can multiply 4×4 matrices.
- **The shape is fixed.** A 4×4 array is efficient for 4×4 problems. Feed it a 3×3 problem and one row and one column of PEs sit idle, doing nothing but burning power.

- **There is no back-pressure.** The design cannot say "wait, I'm busy". The controller must know the exact latency and respect it. That is simpler and faster, but it means a timing mistake is a silent wrong answer rather than a stall.
- **Everything is latency, not choice.** The design's correctness is a set of arithmetic identities about cycle counts. Get one off by one, and it still runs — just wrongly.

6 Every file, line by line

We now walk through the design from the smallest module to the largest. Each module builds on the one before it, so reading in order is worth it.

> **ONE NAMING QUIRK TO KNOW FIRST**
>
> In this repository, eight files have a module inside them whose name is different from the filename. For example the module `processing_element` lives in a file called `MAC.sv`. This is not good practice — most teams enforce one-module-per-file with matching names — but it is what this repository does, so both names are given in every heading below.

6.1 `delay_pipe.sv` — the simplest module

Job: make a value come out `DELAY` clock cycles after it went in. That is all. It is used twelve times in the design and everything else depends on it working.

Think of it as a conveyor belt with `DELAY` slots. Put a value on one end; it appears at the other end `DELAY` cycles later, with everything else on the belt shifting along one slot per cycle.

`design_rtl/delay_pipe.sv` — 33 lines, all of them

```
1 module delay_pipe #(
```

Start of the module. The `#(` opens the **parameter** list — the settings you can change each time you use this module.

```
2     parameter WIDTH = 8,
```

How many wires wide the value is. Default 8. When the de-skew buffer uses this module it passes 32 instead.

```
3     parameter DELAY = 1,
```

How many cycles of delay — how many slots on the conveyor belt. This is the parameter that makes the whole skew mechanism work.

```
4 )(
```

Close the parameter list, open the **port** list — the actual pins of this block of hardware.

```
5     input  logic             clk,
```

The clock. One wire, ticking 0,1,0,1. Every register inside will update on its rising edge.

```
6     input  logic             rst_n,
```

Reset. The `_n` suffix is a universal convention meaning **active low**: the reset is happening when this wire is **0**, not 1.

```
7     input  logic [WIDTH-1:0] din,
```

Data in. `[WIDTH-1:0]` means bits numbered from `WIDTH-1` down to 0 — so 8 wires if `WIDTH` is 8.

```
8     output logic [WIDTH-1:0] dout
```

Data out, same width. No comma — it is the last port.

```
9 );
```

End of the port list. Everything after this is the module's contents.

```
10    generate
```

Start of a **compile-time** region. Code here decides *what hardware gets built*, not what happens at runtime.

```
11        if (DELAY == 0) begin : gen_no_delay
```

If someone asked for zero delay, build the version below. The `: gen_no_delay` gives this branch a name, which is what you will see in a waveform viewer. **Naming generate blocks is a real best practice** — unnamed ones get tool-invented names like `genblk1` that change between tool versions.

```
12            assign dout = din;
```

Zero delay means **literally a piece of wire**. No register, no clock, no reset. The output changes the instant the input does. This branch is not a tolerated edge case — it is used deliberately, by row 0 of the skew buffer and column 3 of the de-skew buffer.

```
13        end
```

End of the zero-delay branch.

```
14        else begin : gen_with_delay
```

Otherwise build a real shift register. Also named.

```
15            logic [WIDTH-1:0] pipe [0:DELAY-1];
```

The conveyor belt: `DELAY` separate storage slots, each `WIDTH` bits wide. The `[0:DELAY-1]` *after* the name makes it an **unpacked array** — a list of separate registers rather than one wide number. This declaration is also why line 11 exists: with `DELAY = 0` this would be a zero-element array, which is illegal.

```
16            integer i;
```

A loop counter used below. It never becomes hardware — it only exists while the tool is elaborating the loops.

```
17
```

Blank line.

```
18            always_ff @(posedge clk) begin
```

"Build registers that update on the rising clock edge." Everything assigned inside this block becomes a flip-flop. `always_ff` also asks the tool to *check* that you really did describe registers, and to complain if you accidentally described something else.

```
19                if (!rst_n) begin
```

"If reset is active" (remember: active low, so `!rst_n` means reset *is* asserted). Because this test is *inside* the `always_ff` and not in the `@(...)` list, this is a **synchronous reset** — it only takes effect on a clock edge.

```
20                    for (i = 0; i < DELAY; i = i +
  1)
```

A loop the tool unrolls at compile time. With `DELAY = 3` it produces three copies of the next line, one per slot.

```
21                        pipe[i] <= '0;
```

Clear every slot to zero. `'0` is shorthand for "all bits zero, however many there are" — safer than writing `8'b0`, which breaks silently if `WIDTH` changes.

```
22                end
```

End of the reset branch.

```
23                else begin
```

Normal operation — not in reset.

```
24                    pipe[0] <= din;
```

The newest value enters slot 0. Note `<=`, the **non-blocking** assignment: it schedules the update for the end of the clock edge rather than doing it immediately.

```
25                    for (i = 1; i < DELAY; i = i +
   1)
```

For every other slot…

```
26                        pipe[i] <= pipe[i-1];
```

…copy from the slot before it. **This is where non-blocking assignment earns its keep.** All the right-hand sides are read using the values from *before* the edge, so every slot shifts along by exactly one. With `=` instead, slot 0's new value would immediately overwrite slot 1, then slot 2, and the whole belt would collapse into one value.

```
27                end
```

End of the normal branch.

```
28            end
```

End of the `always_ff` block.

```
29
```

Blank line.

```
30            assign dout = pipe[DELAY-1];
```

The output is permanently wired to the **last** slot — the end of the conveyor belt, holding the oldest value.

```
31        end
```

End of the with-delay branch.

```
32    endgenerate
```

End of the compile-time region.

```
33 endmodule
```

End of the module.

What this module does, in one picture

```
DELAY = 3

  din ──►| pipe[0] |──►| pipe[1] |──►| pipe[2] |──► dout
         └────────┘    └────────┘    └────────┘
         newest                      oldest

cycle:      1    2    3    4    5    6
din:        7   -3   12    0   25   -8
dout:       0    0    0    7   -3   12      ← din, three cycles later
```

The rule is `dout at cycle m = din at cycle m − DELAY`. See this animated in the technical document, section 6.

6.2 `MAC.sv` — the processing element

Job: hold one weight; multiply every activation that passes through by it; add the result to the running total coming down from above. Sixteen of these make the array, and **every arithmetic operation in the whole chip happens inside one of them**.

"MAC" stands for **Multiply-ACcumulate** — the single most common operation in signal processing and machine learning, and the thing every DSP and every AI chip is built around.

> **THE ONE CLEVER TRICK IN THIS MODULE**
>
> The vertical (north–south) bus is **time-multiplexed**: it carries weights while `wload = 1`, and running totals the rest of the time. This saves building a second 32-wire network across the entire array, at the cost of one multiplexer per PE. Reusing a bus for two purposes in two different phases is a very standard hardware economy.

`design_rtl/MAC.sv` — module processing_element, 64 lines

```
1  `timescale 1ns / 1ps
```

A simulator directive: "one time unit means 1 nanosecond; round to 1 picosecond". It affects simulation only and has no effect on the hardware built.

```
3  module processing_element (
```

Module start. Note: **no parameter list** — this module's widths are hard-coded at 8 and 32. That is a real limitation of the design, and it is the reason the whole project is only verified at 4×4 with int8.

```
4      input  logic               clk,
```

Clock.

```
5      input  logic               rst_n,
```

Active-low synchronous reset.

```
8      input  logic               wload,
```

Mode select. 1 = we are loading weights into the array; 0 = we are computing. This one wire changes what the vertical bus means and what this PE does with it.

```
9      input  logic               valid_in,
```

"The activation arriving this cycle is real data, not a gap." When it is 0, this PE will not accumulate.

```
12     input  logic signed [7:0]  a_in,
```

The activation arriving from the **west** (the PE to the left, or the array boundary). `signed` means −128…+127.

```
13     output logic signed [7:0]  a_out,
```

The same activation leaving to the **east**, one clock later. This is how an activation gets reused by all four PEs in its row.

```
14      output logic               valid_out,
```

The valid flag travelling east alongside `a_out`, delayed by the same one cycle so the two can never separate.

```
17      input  logic signed [31:0] psum_in,
```

From the **north**. During `wload` this carries a weight (in its low 8 bits); otherwise it carries the running total. "psum" = **partial sum**.

```
18      output logic signed [31:0] psum_out
```

To the **south** — either the next weight to shift down, or this PE's updated running total.

```
19  );
```

End of ports. Four inputs of data, three outputs, two control wires, plus clock and reset.

```
22      logic signed [7:0]  weight_q;
```

Register 1 of 4. The resident weight. The `_q` suffix is a long-standing convention meaning "the Q output of a flip-flop", i.e. this is stored state rather than a wire.

```
23      logic signed [7:0]  a_q;
```

Register 2. The activation, held for one cycle so it can be handed east next cycle.

```
24      logic               valid_q;
```

Register 3. The valid flag, delayed to match `a_q`.

```
25      logic signed [31:0] psum_q;
```

Register 4. The running total this PE will hand south. These four registers are the PE's entire state.

```
28      logic signed [31:0] product_ext;
```

A wire (not a register) carrying the multiplier's output.

```
29      logic signed [31:0] adder_out;
```

A wire carrying the adder's output.

```
32      assign product_ext = 32'(weight_q * a_in);
```

The multiplier. Multiply the stored weight by the arriving activation. The `32'(...)` cast forces the result to 32 bits, sign-extended. Without it, SystemVerilog's width rules would give a 16-bit result and the later addition could go wrong. Note this uses `a_in` — the value arriving *now*, not the stored `a_q`.

```
33      assign adder_out   = psum_in + product_ext;
```

The adder. Add this PE's product to the running total coming from the north. Both `assign` statements are pure combinational logic — a real multiplier and a real adder, always live.

```
36      always_ff @(posedge clk) begin
```

Everything below becomes registers clocked on the rising edge.

```
37          if (!rst_n) begin
```

Synchronous reset.

```
38              weight_q <= '0;
41              a_q      <= '0;
                valid_q  <= '0;
                psum_q   <= '0;
```

Clear all four registers. Unlike the memories (which deliberately have no reset), registers here are reset so the array starts in a known, all-zero state.

```
42          end else begin
```

Normal operation.

```
44-             if (wload) begin
46                  weight_q <= psum_in[7:0];
                end
```

Weight capture. Only while `wload` is high, grab the low 8 bits of the vertical bus and store them as this PE's weight. When `wload` is 0 there is no `else`, so `weight_q` simply keeps its value — that is the "stationary" in weight-stationary.

```
49              a_q     <= a_in;
```

The horizontal pipeline: whatever arrived from the west this cycle will be handed to the east next cycle. This one register is why an activation takes one cycle to cross one column.

```
50              valid_q <= valid_in;
```

The valid flag rides along, delayed identically.

```
53              psum_q  <= valid_in ? adder_out :
    psum_in;
```

The accumulator, with a bypass. If the arriving activation is real, store the sum. If not, store the incoming total **unchanged** — do not add garbage to it. Note carefully: even when invalid, the total still *moves down one row*. There is no stalling. Partial sums march south at exactly one row per cycle no matter what, which is why a gap in the input stream can never corrupt a real calculation — the two occupy different pipeline slots at all times.

```
54-         end
55      end
```

End of the else branch and the `always_ff`.

```
58      assign a_out     = a_q;
```

The east output is just the stored activation. Simple wire from a register.

```
59      assign valid_out = valid_q;
```

Same for the valid flag.

```
62      assign psum_out  = wload ? 32'(weight_q) :
    psum_q;
```

The most consequential line in the entire design. During compute it outputs the stored running total — ordinary. But during `wload` it outputs `weight_q` *combinationally*, skipping the register. That means the PE below sees this PE's current weight immediately, and captures it on the next edge. A whole column therefore behaves as a **downward shift register**, and **the weight fed in first travels the furthest**. See the box below.

```
64  endmodule
```

End.

Why weights must be loaded bottom row first

Because of line 62, feeding four weight values `w₁ w₂ w₃ w₄` into the top of a column on four consecutive cycles produces this:

```
After edge  | PE row 0 | PE row 1 | PE row 2 | PE row 3
     1      |    w₁    |    –     |    –     |    –
     2      |    w₂    |    w₁    |    –     |    –
```

```
      3      |    W₃    |    W₂    |    W₁    |    –
      4      |    W₄    |    W₃    |    W₂    |    W₁   ← W₁ travelled furthest
```

RULE YOU MUST NOT BREAK

To get weight row *r* into array row *r*, feed **W[3] first**, then W[2], W[1], W[0]. The address generator does this by counting the weight address **down**: 3, 2, 1, 0.

Getting this wrong is completely silent. The design still runs, the "done" signal still fires, and plausible-looking numbers still appear in memory. They are simply the answer to a *different* question — the weight matrix flipped upside down. Nothing detects it. This is why every test in this project uses a **random, non-symmetric** weight matrix: with a symmetric or identity matrix, flipping the rows would produce the same answer and the bug would pass every test.

6.3 `array.sv` — wiring 16 PEs together

Job: create 16 processing elements and connect them into a grid. This module contains **no arithmetic at all** — it only names wires. That is a deliberate and very common structuring choice: keep computation in leaf modules, keep interconnect in structural modules, and never mix the two.

`design_rtl/array.sv` — module systolic_array, key lines

3–6
```
module systolic_array #(
    parameter ROWS = 4,
    parameter COLS = 4
)(
```

Grid dimensions as parameters, so the same source could build an 8×8 array. (In practice the PE's hard-coded widths limit this — see the limitations section.)

14
```
    input logic signed [7:0] a_in_left [0:ROWS-1],
```

The **west boundary**: four separate 8-bit inputs, one per row. The `[0:ROWS-1]` after the name makes it an **unpacked array** — four independent buses, not one 32-bit number.

15
```
    input logic valid_in_left [0:ROWS-1],
```

One valid flag per row. Note that in this design all four are always driven with the same value — per-row valid is supported by the hardware but never actually used.

18
```
    input logic signed [31:0] psum_in_top [0:COLS-
1],
```

The **north boundary**: four 32-bit inputs, one per column. Carries weight values during load, and zeros during compute (because every dot product must start from 0).

21
```
    output logic signed [31:0] psum_out_bottom
[0:COLS-1]
```

The **south boundary**: the finished dot products leaving the array.

```
25-     logic signed [7:0]  a_net     [0:ROWS-1]
27 [0:COLS];
        logic               valid_net [0:ROWS-1]
   [0:COLS];
        logic signed [31:0] psum_net  [0:ROWS]
   [0:COLS-1];
```

The three interconnect meshes. Look carefully at the sizes: `a_net` has `COLS+1` columns, and `psum_net` has `ROWS+1` rows — one more than the grid, in the direction the data travels. That extra slot holds the boundary, which means feeding an input is just an ordinary `assign` rather than a special case. It is a small trick that removes a lot of awkward code.

```
29      genvar r, c;
```

Compile-time loop variables. `genvar` (not `integer`) is required for loops that build hardware.

```
32-     for (r = 0; r < ROWS; r++) begin : left_bind
35          assign a_net[r][0]     = a_in_left[r];
            assign valid_net[r][0] = valid_in_left[r];
        end
```

Wire the west inputs into column 0 of the mesh. Because of the oversized mesh, this is four plain wires and nothing more.

```
37-     for (c = 0; c < COLS; c++) begin : top_bind
39          assign psum_net[0][c] = psum_in_top[c];
        end
```

Same for the north inputs into row 0.

```
42-     for (r = 0; r < ROWS; r++) begin : row_gen
43          for (c = 0; c < COLS; c++) begin : col_gen
```

The nested generate loop that builds the grid. This does not "run" — it instructs the tool to stamp out 4 × 4 = 16 physical copies of the PE. In a waveform viewer, PE row 2 column 3 will be called `row_gen[2].col_gen[3].pe_inst`.

```
50-             .valid_in  (valid_net[r][c]),
53              .a_in      (a_net[r][c]),
                .valid_out (valid_net[r][c+1]),
                .a_out     (a_net[r][c+1]),
```

The horizontal link. Each PE reads from mesh position `[r][c]` and writes to `[r][c+1]` — so PE *c*'s output is automatically PE *c*+1's input. The chain builds itself; there is no wiring list to get wrong.

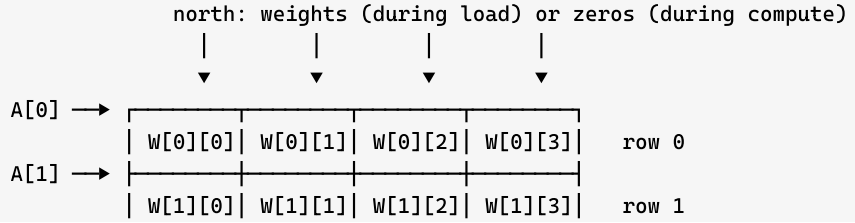
```
56-             .psum_in  (psum_net[r][c]),
57              .psum_out (psum_net[r+1][c])
```

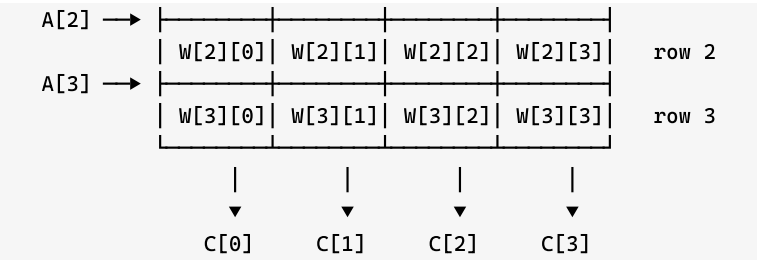
The vertical link. Identical idea in the other direction: read from row *r*, write to row *r*+1.

```
63-     for (c = 0; c < COLS; c++) begin : bottom_bind
65          assign psum_out_bottom[c] = psum_net[ROWS]
   [c];
        end
```

Export the extra row of the mesh — row `ROWS`, which no PE writes to except as an output — as the array's south boundary.

The resulting picture





6.4 `input_buf.sv` — the skew buffer

Job: delay row *r*'s activation by *r* cycles, turning a "all four numbers at once" vector into a diagonal staircase.

Why this is necessary

An activation entering row 0 reaches column 3 after three cycles of travel. Meanwhile the running total in column 3 descends from row 0 to row 3, also taking cycles. For the sum arriving at the bottom of column 3 to contain four products **from the same activation vector**, the four elements have to arrive at the array's west edge at staggered times — element *r* exactly *r* cycles after element 0.

If you fed all four in at once instead, the sum leaving the bottom of a column would mix element 0 of one vector with element 1 of the previous vector, element 2 of the one before that, and so on. The design would still produce numbers. They would just be meaningless.

`design_rtl/input_buf.sv` — module input_skew_buffer, 36 lines

```
1–4  module input_skew_buffer #(
         parameter ROWS  = 4,
         parameter WIDTH = 8
     )(
```

Parameters: how many rows, and how wide each activation is.

```
7–8      input logic signed [WIDTH-1:0] dense_a_in
     [0:ROWS-1],
         input logic                    dense_valid_in
     [0:ROWS-1],
```

"Dense" means **un-staggered** — all four elements of one vector, presented on the same cycle. This is what memory naturally gives you.

```
9–
10       output logic signed [WIDTH-1:0] skew_a_out
     [0:ROWS-1],
         output logic                    skew_valid_out
     [0:ROWS-1]
```

"Skewed" means staggered — what the array actually needs.

```
14       for (r = 0; r < ROWS; r = r + 1) begin :
     gen_input_skew
```

Build one delay path per row.

```
15–23         delay_pipe #(
                  .WIDTH(WIDTH),
                  .DELAY(r)
              ) u_a_delay (
                  .clk(clk), .rst_n(rst_n),
                  .din (dense_a_in[r]),
                  .dout(skew_a_out[r])
              );
```

The whole idea, in one instantiation. `.DELAY(r)` — the delay is the row number. Row 0 gets `DELAY = 0` (a plain wire), row 1 gets one register, row 3 gets three. That is why `delay_pipe` had to handle `DELAY = 0` properly: it is used here, in the most ordinary way possible.

```
25–33         delay_pipe #(
                  .WIDTH(1),
                  .DELAY(r)
              ) u_v_delay (
                  .clk(clk), .rst_n(rst_n),
                  .din (dense_valid_in[r]),
                  .dout(skew_valid_out[r])
              );
```

A **second** pipe, one bit wide, for the valid flag — with the *same* delay. This matters: the PE decides whether to accumulate by looking at `valid_in` in the same cycle it consumes `a_in`. If data and valid got out of step by even one cycle, the array would accumulate garbage or skip real data.

```
cycle:        1     2     3     4     5     6
dense in:    A0    A1    A2     –     –     –      (whole vectors)

west edge, after skewing:
  row 0 →   A0[0] A1[0] A2[0]    –     –     –     delay 0
  row 1 →     –   A0[1] A1[1] A2[1]    –     –     delay 1
  row 2 →     –     –   A0[2] A1[2] A2[2]   –      delay 2
  row 3 →     –     –     –   A0[3] A1[3] A2[3]    delay 3
                            ↖ A0 now forms a diagonal
```

6.5 `output_skewbuf.sv` — the de-skew buffer

Job: undo the staircase on the way out. Column *c* is delayed by `COLS − 1 − c`.

Column 0 finishes first (its activations had the shortest journey east), so it is delayed the most — 3 cycles. Column 3 finishes last, so it is not delayed at all. The two effects cancel exactly, and all four results emerge on the **same** cycle.

`design_rtl/output_skewbuf.sv` — module output_deskew_buffer, 24 lines

```
7–8       input  logic signed [WIDTH-1:0] psum_in
      [0:COLS-1],
          output logic signed [WIDTH-1:0] aligned_out
      [0:COLS-1]
```

In: the staircase straight off the bottom of the array. Out: all four lanes aligned. Here `WIDTH` is 32 — these are full 32-bit results, not 8-bit activations.

```
13–16       delay_pipe #(
                .WIDTH(WIDTH),
                .DELAY(COLS-1-c)
            ) u_psum_delay (
```

The mirror of the input skew. `COLS-1-c` gives 3, 2, 1, 0 for columns 0, 1, 2, 3 — the exact reverse of the input side. Column 3's instance has `DELAY = 0` and is a plain wire.

```
The cancellation, written out:

  column c result leaves the array at   n + (ROWS-1) + c
  de-skew adds                                  + (COLS-1-c)
                                        ──────────────────────
  arrives at                            n + ROWS + COLS - 2        ← no "c" left!

  For 4×4:  n + 6, for every column.
```

Because the answer no longer depends on *c*, the four lanes can simply be glued together into one 128-bit memory word. That is the entire reason this module exists.

6.6 The three memories

`a_mem.sv`, `weight_mem.sv` and `outmem.sv` are the **same module three times**, with different sizes. Once you understand one, you understand all three.

`design_rtl/a_mem.sv` — 21 lines; weight_mem.sv and outmem.sv are identical apart from sizes

```
3–6 module a_mem #(
        parameter DATA_W = 32,
        parameter ADDR_W = 8,
        parameter DEPTH  = (1 << ADDR_W)
    )(
```

`DATA_W` = how many bits per word. `ADDR_W` = how many address wires. `DEPTH` is *derived*: `1 << 8` is 1 shifted left 8 times = 256. Deriving it means the address width and the depth can never disagree.

```
8       input  logic              clk,
```

Clock — this is a **synchronous** memory.

```
9       input  logic              we,
```

Write enable. 1 = write this cycle, 0 = do not.

```
10      input  logic [ADDR_W-1:0] addr,
```

One address, shared by reads and writes. This is a **single-port** memory — there is only one way in, so you cannot read one location while writing another. This single fact is what forces the arbitration logic at the top of the design.

```
11      input  logic [DATA_W-1:0] wdata,
```

Data to write.

```
12      output logic [DATA_W-1:0] rdata
```

Data read back.

```
14    logic [DATA_W-1:0] mem [0:DEPTH-1];
```

The storage itself: 256 separate words of 32 bits. In simulation this is an array; in real silicon the synthesis tool replaces it with a purpose-built SRAM block.

```
16    always_ff @(posedge clk) begin
```

Everything happens on the clock edge.

```
17-       if (we)
18            mem[addr] <= wdata;
```

If write enable is high, store the data at the given address.

```
19        rdata <= mem[addr];
```

Three separate things are happening here, all of them deliberate.
(1) The read is *outside* the `if (we)`, so it happens every single cycle. Guarding it would infer a "read enable" that many real memory blocks do not have.
(2) It uses `<=`, so `rdata` is a register: data comes back **one cycle after** the address is presented. This one-cycle read latency is the reason the design has to re-time its control signals.
(3) Because both assignments are non-blocking, reading and writing the same address on the same cycle returns the **old** value — "read-before-write" behaviour.

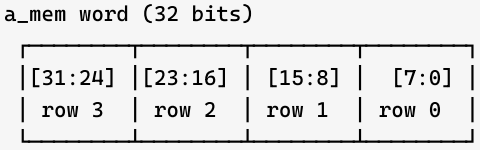
WHY THE CODE LOOKS EXACTLY LIKE THIS

This is the **canonical inference template** for a single-port synchronous RAM. Every FPGA and ASIC synthesis tool recognises this exact shape and maps it onto a dedicated memory block — a Block RAM on an FPGA, a compiled SRAM macro on an ASIC. Write it slightly differently and the tool may build the memory out of thousands of individual flip-flops instead, which can be 20–50× larger and slower.

Notice also there is **no reset**. That is intentional: real SRAM macros physically cannot be cleared in one cycle, so adding a reset would prevent the tool from using one. The trade-off is that reading an address you never wrote returns `X` ("unknown") in simulation — which is arguably a feature, because it makes the mistake visible instead of silently returning zero.

What each memory holds

Memory	Size	One word contains
`a_mem`	256 words × 32 bits	Four int8 activations — one activation vector. Lane *r* (bits `[r*8+7 : r*8]`) feeds array **row** *r*.
`weight_mem`	4 words × 32 bits	Four int8 weights — one row of W. Lane *c* feeds array **column** *c*. Address *r* holds weight row *r*.
`output_mem`	256 words × 128 bits	Four int32 results — one result vector. Lane *c* is the output of array **column** *c*.



```
output_mem word (128 bits)
 ┌──────────┬──────────┬──────────┬──────────┐
 │ [127:96] │ [95:64]  │ [63:32]  │ [31:0]   │
 │  col 3   │  col 2   │  col 1   │  col 0   │
 └──────────┴──────────┴──────────┴──────────┘
```

6.7 `controller.sv` — the state machine

Job: run the job through its phases, in order, for the right number of cycles each. It is the conductor of the orchestra.

A **finite state machine** (FSM) is the standard way to build sequential control in hardware. It is a register holding "which state am I in", plus rules for when to move to the next state. Almost every piece of control logic you will ever see in hardware is an FSM.

> **THE DIVISION OF LABOUR**
>
> The controller owns **durations only** — how many cycles each phase lasts. It never produces an address. The address generator (next section) owns **addresses only** and never decides when a phase starts or ends. Neither one knows that the memories have a one-cycle read latency.
>
> Splitting responsibilities this cleanly is what allows each to be tested completely on its own, and it is why the `agu.sv` file contains no timing constants at all.

The six phases

State	How long	What is happening
`IDLE`	Until `start`	Doing nothing. Waiting for the host to say go.
`WLOAD`	4 cycles (= `ROWS`)	Shifting the four weight rows into the array, bottom row first.
`FLUSH`	6 cycles (= `ROWS+2`)	Pushing zeros through to clear out weight values that got stuck in the accumulator registers during WLOAD.
`COMPUTE`	`num_vectors` cycles	The real work: one activation vector per clock.
`DRAIN`	8 cycles (= `STORE_LATENCY`)	The last vector is still travelling through the pipeline. Wait for it to come out and be stored.
`DONE`	1 cycle	Pulse the `done` signal, then return to IDLE.

Total job length: `4 + 6 + num_vectors + 8 + 1 = num_vectors + 19` cycles.

`design_rtl/controller.sv` — module controller, the logic (comments omitted)

```
39     parameter int FLUSH_CYCLES = ROWS + 2,
```

The flush length, *derived* from the array size rather than hard-coded as 6. Change `ROWS` and this follows automatically.

```
43      parameter int STORE_LATENCY = 1 + ROWS + COLS
   - 2 + 1
```

The single most important number in the design, written as its derivation rather than as "8": one cycle of memory read latency, plus `ROWS+COLS-2` cycles through the array and buffers, plus one cycle for the write strobe to line up. The same expression appears in the AGU and in the wrapper, so all three always agree.

```
64-     localparam logic [2:0] PH_IDLE    = 3'd0;
69      localparam logic [2:0] PH_WLOAD   = 3'd1;
        localparam logic [2:0] PH_FLUSH   = 3'd2;
        localparam logic [2:0] PH_COMPUTE = 3'd3;
        localparam logic [2:0] PH_DRAIN   = 3'd4;
        localparam logic [2:0] PH_DONE    = 3'd5;
```

Names for the six states. `localparam` is a constant that *cannot* be overridden from outside — the right choice for internal encodings. Six states need 3 bits (`3'd0` means "3 bits wide, decimal value 0").

```
71-     logic [2:0]           state;
73      logic [VEC_CNT_W-1:0] cnt;
        logic [VEC_CNT_W-1:0] vec_n;
```

Three registers, and that is the entire state of the controller: which phase we are in, how many cycles we have been in it, and how many vectors this job asked for.

```
83-     PH_IDLE: begin
89          cnt <= '0;
            if (start) begin
                vec_n <= num_vectors;
                state <= PH_WLOAD;
            end
        end
```

In IDLE, keep the counter cleared. When `start` arrives, **latch** `num_vectors` into `vec_n` and move to WLOAD. Latching matters: the host is free to change `num_vectors` immediately afterwards without affecting the running job.

```
91-     PH_WLOAD: begin
97          if (cnt == VEC_CNT_W'(ROWS-1)) begin
                cnt   <= '0;
                state <= PH_FLUSH;
            end
            else cnt <= cnt + 1'b1;
        end
```

The pattern used by every phase: count up; when the counter reaches "length − 1", reset it and move on. Counting to `ROWS-1` starting from 0 gives exactly `ROWS` cycles in this state. The `VEC_CNT_W'(...)` cast makes the widths match explicitly, avoiding a class of silent comparison bug.

```
99-     PH_FLUSH: begin
106         if (cnt == VEC_CNT_W'(FLUSH_CYCLES-1))
    begin
                cnt   <= '0;
                state <= (vec_n == '0) ? PH_DRAIN :
    PH_COMPUTE;
            end
            else cnt <= cnt + 1'b1;
        end
```

A one-line guard that prevents a disaster. If the host asked for zero vectors, skip COMPUTE entirely. Without it, COMPUTE would test `cnt == vec_n - 1` = `0 - 1` = 255, and the machine would sit there for 256 cycles issuing garbage addresses and 256 write strobes, actively corrupting memory. One comparator turns a malformed command from destructive into harmless.

```
108–114     PH_COMPUTE: begin
                if (cnt == (vec_n - 1'b1)) begin
                    cnt   <= '0;
                    state <= PH_DRAIN;
                end
                else cnt <= cnt + 1'b1;
            end
```

Stay in COMPUTE for exactly `vec_n` cycles — one activation vector per cycle.

```
116–122     PH_DRAIN: begin
                if (cnt == VEC_CNT_W'(STORE_LATENCY-1))
    begin
                    cnt   <= '0;
                    state <= PH_DONE;
                end
                else cnt <= cnt + 1'b1;
            end
```

Wait exactly as long as the pipeline is deep, so the last result has time to emerge and be written.

```
129         default: state <= PH_IDLE;
```

If the state register somehow held an illegal value (states 6 or 7 are unused), go back to IDLE. This is standard defensive practice: it guarantees the machine can never get permanently stuck.

```
134–140     assign phase         = state;
            assign wload_phase   = (state == PH_WLOAD);
            assign flush_phase   = (state == PH_FLUSH);
            assign compute_phase = (state == PH_COMPUTE);
            assign drain_phase   = (state == PH_DRAIN);
            assign busy   = (state != PH_IDLE) && (state
    != PH_DONE);
            assign done   = (state == PH_DONE);
```

All outputs are **pure functions of the state register**. This makes it a **Moore machine**, which is the safer of the two standard FSM styles — the outputs cannot glitch in response to a changing input, because they do not depend on inputs at all. Note that `busy` is already *low* when `done` pulses, which is exactly what lets the host start reading results on the very cycle it sees `done`.

6.8 `agu.sv` — the address generator

Job: for each phase, produce the memory addresses that phase needs. AGU stands for **Address Generation Unit** — a standard term you will meet in CPUs, DMA engines and every kind of accelerator.

`design_rtl/agu.sv` — module agu, the logic

```
75      logic [W_ADDR_W-1:0] w_cnt;
```

A 2-bit counter for the weight address. Two bits = values 0…3, which is exactly the four weight rows.

```
77–81     always_ff @(posedge clk) begin
              if (!rst_n)            w_cnt <=
          W_ADDR_W'(ROWS-1);
              else if (!wload_phase) w_cnt <=
          W_ADDR_W'(ROWS-1);
              else                   w_cnt <= w_cnt -
          1'b1;
          end
```

Counting down: 3, 2, 1, 0. This is the RTL implementation of the bottom-first weight loading rule from section 6.2. Two details worth noticing:
(1) The counter is **re-armed to 3 whenever we are not in WLOAD**, not just on reset. That means the very first WLOAD cycle already presents 3, and a second job starts from 3 again rather than continuing from where the last one stopped.
(2) It is allowed to wrap past 0 — but by then WLOAD is over and the counter is re-armed anyway.

```
83–85     assign w_addr  = w_cnt;
          assign w_rd_en = wload_phase;
          assign wload   = wload_phase;
```

The address is the counter; the read enable and the `wload` mode signal are simply the phase itself. Note `wload` is issued in the **same** cycle as its address — the wrapper delays it internally to match memory latency.

```
92–96     always_ff @(posedge clk) begin
              if (!rst_n)              act_cnt <= '0;
              else if (!compute_phase) act_cnt <= '0;
              else                     act_cnt <=
          act_cnt + 1'b1;
          end
```

The activation counter, counting **up** this time: 0, 1, 2, … Same re-arm-while-idle pattern, so every job starts from 0.

```
98        assign act_addr = act_base + act_cnt;
```

The actual address is a programmable **base** plus the counter. That is what lets the host store several different batches of activations in the same memory and choose which one to run.

```
106–107   logic                out_we_pipe
          [0:STORE_LATENCY-1];
          logic [O_ADDR_W-1:0] out_addr_pipe
          [0:STORE_LATENCY-1];
```

Two 8-deep shift registers — one carrying a write-enable bit, one carrying an address.

```
117–122           out_we_pipe[0]   <= compute_phase;
                  out_addr_pipe[0] <= out_base +
          O_ADDR_W'(act_cnt);
                  for (int i = 1; i < STORE_LATENCY; i++)
          begin
                      out_we_pipe[i]   <= out_we_pipe[i-1];
                      out_addr_pipe[i] <= out_addr_pipe[i-
          1];
                  end
```

The nicest design decision in the project. When a result will be ready, it is 8 cycles in the future. The obvious solution is to compute "assert the write strobe when the counter reaches n + 8". Instead, the AGU pushes the pair {write-enable, address} onto a conveyor belt exactly 8 slots long. It falls off the far end precisely when the data arrives.

Why this is better: the strobe **structurally cannot** drift away from its data. If the pipeline depth ever changes, you change one parameter and everything follows — instead of re-deriving an equation and hoping you found every place that used the old number. The cost is 8 flip-flops. That is a trade almost any hardware engineer would take.

```
126-     assign out_we   = out_we_pipe[STORE_LATENCY-
127 1];
        assign out_addr = out_addr_pipe[STORE_LATENCY-
    1];
```

The end of the belt drives the output memory's write port.

6.9 The three wrappers

A **wrapper** (also called a top-level or integration module) is a module whose job is to instantiate other modules and connect them. The design has three, one per level of completeness.

> **THE RULE THIS PROJECT FOLLOWS**
>
> **Each level instantiates the level below it** — it does not re-create the same wiring. So the datapath tested at level 2 is *literally the same hardware* running inside level 3. They cannot drift apart, and a bug that appears only at level 3 must therefore be in the control logic that level 3 adds.
>
> And: **no wrapper file ever overwrites another.** Adding a fourth level means adding a fourth file, not editing the third.

Level 1 — `wrapper_array_buf.sv` (module `array_buf_top`)

Skew buffer + array + de-skew buffer. No memory, no sequencing: the caller hands it one dense vector per clock and reads the result back 6 cycles later. It is three instantiations and one constant:

```
localparam int RESULT_LATENCY = ROWS + COLS - 2;   // = 6, stated for testbenches

input_skew_buffer    u_input_skew    ( dense_a_in  -> skew_a  );
systolic_array       u_array         ( skew_a      -> psum_raw );
output_deskew_buffer u_output_deskew ( psum_raw    -> result   );
```

It also exports `psum_raw` — the results *before* de-skewing. Nothing uses it. It exists purely so a testbench can look at the staircase directly and confirm it is real. Deliberately exposing internal signals for observability is standard practice, and it costs nothing.

Level 2 — `wrapper_mem_arr_buf.sv` (module `accelerator_top`)

Adds the three memories, the control re-timing, and the packing/unpacking of memory words.

`wrapper_mem_arr_buf.sv` — the four pieces of logic it adds

```
94–     always_ff @(posedge clk) begin
103         if (!rst_n) begin
                wload_q <= 1'b0;
                valid_q <= 1'b0;
            end else begin
                wload_q <= wload;
                valid_q <= valid_in;
            end
        end
```

The re-timing. The memories take one cycle to return data, so the control signals must be delayed by one cycle to line up with it. Two registers do it.

Why here and not at the caller? Because it makes the rule the user sees uniform and hard to get wrong: **issue control in the same cycle as its address.** Everything else in the design follows that one rule, and no other module needs to know about memory latency.

```
147–    always_comb begin : unpack_activations
152         for (int r = 0; r < ROWS; r++) begin
                dense_a[r] = signed'(a_rdata[r*DATA_W
    +: DATA_W]);
                dense_v[r] = valid_q;
            end
        end
```

Unpacking. Split the 32-bit memory word into four separate signed 8-bit values, one per array row. `signed'(...)` is essential — without it the tool treats the bits as unsigned and a weight of −5 becomes +251.

Note `dense_v[r] = valid_q` for every *r*: one valid bit broadcast to all four rows. Per-row valid is supported by the hardware but never used.

```
159–    always_comb begin : drive_array_top
163         for (int c = 0; c < COLS; c++)
                psum_top[c] = wload_q ?
    PSUM_W'(signed'(w_rdata[c*DATA_W +: DATA_W]))
                                      : '0;
        end
```

The north edge multiplexer. During weight load, feed sign-extended weights in. Otherwise feed **zeros** — because every dot product must start from zero, and the top of each column is where it starts. The double cast `PSUM_W'(signed'(…))` first says "these 8 bits are signed", then "widen to 32 bits" — in that order, so −5 becomes 32 bits of −5 rather than 32 bits of 251.

```
190–    always_comb begin : pack_output
193         for (int c = 0; c < COLS; c++)
                o_wdata[c*PSUM_W +: PSUM_W] =
    result[c];
        end
```

Packing. Glue the four aligned 32-bit results into one 128-bit memory word. This is the "Output Processing" block of the architecture diagram — for this design that is exactly lane packing, nothing more. It only works because the de-skew buffer already aligned all four lanes onto the same cycle.

Level 3 — `wrapper_full_system.sv` (module `accelerator_system_top`)

The complete design. It adds the controller, the AGU, and one piece of logic that has not appeared yet: **port arbitration**.

`wrapper_full_system.sv` — the arbitration block

```
164–176     always_comb begin : port_arbitration
                if (busy) begin
                    a_we_mux    = 1'b0;
                    a_addr_mux  = agu_act_addr;
                    a_wdata_mux = '0;

                    w_we_mux    = 1'b0;
                    w_addr_mux  = agu_w_addr;
                    w_wdata_mux = '0;

                    o_we_mux    = agu_out_we;
                    o_addr_mux  = agu_out_addr;
                end
```

While a job is running, the AGU owns all three memory ports. Because each memory has only one address port (section 6.6), exactly one master can drive it — so this multiplexer is not optional, it is forced by the memory design.

Notice that the host's write enables are hard-wired to `1'b0` here, not simply ignored. That means a stray host write during a job **cannot** corrupt anything. This is verified adversarially: one test hammers `0xDEADBEEF` into the weight write port on every single cycle of a live job and requires the results to be bit-identical to a clean run.

```
177–188         else begin
                    a_we_mux    = host_a_we;
                    a_addr_mux  = host_a_addr;
                    a_wdata_mux = host_a_wdata;
                    ...
                    o_we_mux    = 1'b0;
                    o_addr_mux  = host_o_addr;
                end
            end
```

While idle, the host owns the ports and can load matrices or read results. Note `o_we_mux = 1'b0` even here: the host side of the output memory is **read-only**, because only the datapath should ever write results.

7 Running a job from start to finish

Here is the complete life of one job, in plain language.

1. **Reset.** Hold `rst_n` low for a few cycles, then release it. All registers clear to zero. The memories do *not* clear — they hold whatever they held, or unknown values if never written.

2. **Load the weights.** While `busy` is low, write four words into `weight_mem`: address 0 gets weight row 0, address 1 gets row 1, and so on. Each word packs four int8 weights, one per array column.

3. **Load the activations.** Still while idle, write your activation vectors into `a_mem`, starting at whatever address you plan to use as `act_base`. Each word packs four int8 values, one per array row.

4. **Start.** Pulse `start` high for one cycle, with `num_vectors`, `act_base` and `out_base` valid at the same time. The controller latches `num_vectors` and moves to WLOAD. `busy` goes high.

5. **Weight load (4 cycles).** The AGU counts the weight address *down*: 3, 2, 1, 0. Each weight row is read from memory, sign-extended, driven onto the array's north edge, and shifted

down one row per cycle. After four cycles every PE holds its correct weight.

6 **Flush (6 cycles).** Zeros are pushed through with `valid` low, clearing weight residue out of the accumulator registers so the result bus is visibly quiet before real work begins.

7 **Compute (`num_vectors` cycles).** One activation vector is read per cycle, split into four int8 values, staggered by the skew buffer, and fed into the array. Sixteen multiplies and sixteen adds happen every single cycle.

8 **Results emerge and are stored.** Seven cycles after each activation address was issued, its four results appear together on the result bus. One cycle after that — 8 cycles after the address — the AGU's write strobe reaches the end of its conveyor belt and the 128-bit result word is written to `output_mem`.

9 **Drain (8 cycles).** The last vector is still in the pipeline. Wait exactly as many cycles as the pipeline is deep.

10 **Done.** `busy` goes low and `done` pulses for exactly one cycle. Read your results back through `host_o_addr` / `o_rdata` — one 128-bit word per result vector, four int32 values per word.

THINGS THAT ARE SAFE TO DO

Pulsing `start` while a job is already running does **nothing** — it is only sampled in IDLE. Asking for `num_vectors = 0` is legal and completes normally in 19 cycles. Writing to memory during a job is masked off and cannot corrupt anything. All three of these are explicitly tested.

THINGS THAT ARE NOT SAFE TO DO

There is **no back-pressure**. The design will never tell you it is not ready. If you read the output memory before `done`, you will get whatever happens to be there. If you computed the latency wrong and stored a result one cycle early, the design will not object — it will simply store the wrong number, and only a comparison against an independently computed expected value will ever notice.

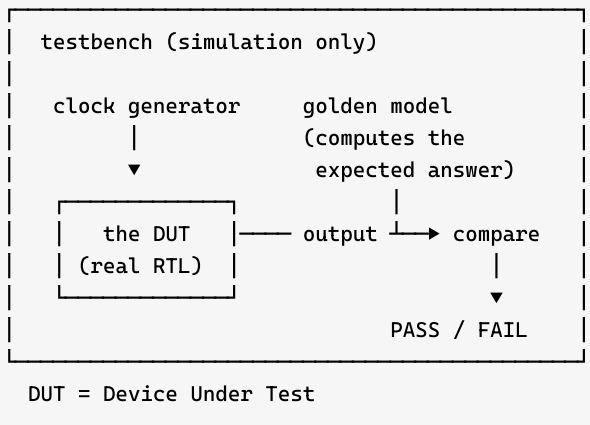
8 How the design is tested

Hardware cannot be debugged after it is manufactured. A chip costs millions of dollars and months of time to fabricate, and once it exists you cannot patch it. So verification is not a phase that happens at the end — in industry it typically consumes **more effort than the design itself**, often by a factor of two or three.

What a testbench is

A **testbench** is a piece of SystemVerilog that is *not* meant to become hardware. It creates a fake clock, wiggles the inputs of the module under test, watches the outputs, and compares them

against what they should be. It runs in a **simulator** — here, QuestaSim.



The golden model idea

The critical question in verification is: *where does the expected answer come from?*

If you write the expected answer by copying the design's own logic, you have proved only that two copies of the same idea agree — and if the idea is wrong, both are wrong together. So from the array level upwards, this project computes the expected answer **from the mathematical definition**:

```
for (int c = 0; c < COLS; c++) begin
    golden[i][c] = '0;
    for (int r = 0; r < ROWS; r++)
        golden[i][c] = golden[i][c] + 32'(A[i][r] * W[r][c]);
end
```

Nothing in that loop knows about skew buffers, pipeline depth, weight shift direction or lane packing. It is the **specification, written as arithmetic**. If the design disagrees with it, the design is wrong — full stop.

Negative checks: the thing most beginners miss

A test that only checks "the right answer appeared" can be passed by a broken design. Suppose the result actually appears one cycle early — a test that just waits and then looks will still find the right number.

So this project also checks things that must **not** happen:

Negative check	What it would catch
The result is **not** present one cycle early	A wrong pipeline depth that happens to still produce the right value
The de-skew output is **empty** until the last column is fed	A de-skew delay that is too short

Negative check	What it would catch
A word past the end of a short run is **untouched**	A write strobe that fires one time too many
The total number of write strobes equals the vector count **exactly**	A pipeline that drops one strobe and emits a spurious one elsewhere
`start` during a job changes **nothing**	An FSM that can be knocked out of sequence
The array is **all zero** after the flush	Weight residue leaking into real results

The results

Metric	Value
Testbenches	13 — one per module, never two modules in one file
Individual checks	3 902
Passing	13/13, 0 mismatches
Defects found	3 — **all three were bugs in the testbenches, not the design**

Two of those three defects were the same underlying mistake: **sampling a signal after the clock edge that consumes it**. If you look at a wire just after an edge, a plain piece of wire and a one-cycle register look identical — both show the value that was there before the edge. You have to look *before* the edge to tell them apart. That is why every testbench in this project states, in its header comment, which of two sampling conventions it uses.

> **WHAT INDUSTRY DOES BEYOND THIS**
>
> This project's approach — directed tests plus algorithmic golden models plus negative checks — is solid, and it is what a small team or a student project would do. A production chip adds: **constrained-random verification** (generate millions of legal random stimuli rather than hand-picked ones), **functional coverage** (measure which behaviours were actually exercised, so "all tests pass" becomes a measured number rather than a hope), **assertions** (SVA properties checked continuously throughout simulation rather than at chosen moments), **UVM** (a standard library for building reusable, layered testbenches), and **formal verification** (mathematically proving a property holds for *all* inputs rather than testing a sample).
>
> This project documents honestly that it has none of those. "13/13 passing" is a pass rate, not a coverage number — and knowing the difference is itself an important piece of the skill.

9 Architecture choices & what industry actually does

Every design is a pile of decisions. Here are the ones that shaped this project, each with the alternative that was rejected and why.

9.1 Weight-stationary dataflow

Chosen because the same weights are reused across every activation vector in a job, so keeping them put eliminates almost all memory traffic.
Alternatives: *output-stationary* (each PE owns one output element and accumulates in place — better when there are very many inputs per output) and *row-stationary* (used by the MIT Eyeriss chip, optimised for convolutions where a filter row is reused in a specific pattern).
Industry: weight-stationary is what Google's TPU v1 used, at 256×256 instead of 4×4.

9.2 int8 operands, int32 accumulator

Chosen because an 8-bit multiplier is dramatically smaller and cheaper than a floating-point one, and neural networks tolerate the precision loss.
Industry: this is **quantisation**, and it is now completely standard for inference. Frameworks like TensorFlow Lite, ONNX Runtime and PyTorch all ship int8 quantisation paths. Newer accelerators push further to int4 and various 8-bit floating-point formats.

9.3 A fixed pipeline with no back-pressure

Chosen because the latency is known exactly and never varies, so a handshake would cost logic and buy nothing.
Alternative: a ready/valid handshake (the AXI-Stream style used almost everywhere in SoC design), which lets blocks stall each other and makes systems composable.
Trade-off: the fixed pipeline is smaller and faster, but any latency mistake is a silent wrong answer instead of a stall. For a fixed-function datapath this is the normal choice; for anything that talks to a bus or a variable-latency memory it would not be.

9.4 Three abstraction levels in three files

Chosen because each level can be verified independently, and a failure at level 3 must therefore be in the control logic that level 3 adds.
Industry: this is standard hierarchical design and hierarchical verification. Real projects go further with formal interface contracts and separately signed-off IP blocks, but the principle — verify the leaves, then verify the integration, and never re-wire what you already verified — is exactly the same.

9.5 The write strobe as a shift register, not an equation

Chosen because it makes the strobe structurally unable to drift from its data.
Alternative: compute the target cycle arithmetically, which costs fewer flip-flops.
Principle: prefer a structural guarantee to a computed one. An equation can be right today and wrong after someone adds a pipeline stage; a shift register of the right length simply follows. This is one of the most transferable ideas in the whole project.

9.6 Synchronous, active-low reset

Chosen because it is the house style throughout: `rst_n` is tested inside the `always_ff` rather than in the sensitivity list.
Industry: both synchronous and asynchronous reset are common; the important thing is *consistency*. Active-low naming (`_n`) is near-universal because in CMOS it is cheaper to drive a signal low reliably, and an undriven line naturally pulls high — so a broken connection asserts reset rather than releasing it.

9.7 Parameters everywhere, derived constants never duplicated

`STORE_LATENCY = 1 + ROWS + COLS - 2 + 1` appears in the controller, the AGU and the wrapper — but always as that expression, never as the literal 8. Change the array size and all three follow.
Industry: this is the **single source of truth** principle, and it is one of the clearest markers of professional RTL. The most common real-world bug pattern in accelerators is a magic number that got updated in three places out of four.

9.8 Things this project deliberately does not do

- **No synthesis or timing closure.** The design has never been run through a tool that turns it into actual gates and checks that it can hit a target clock frequency. Simulation says the logic is right; it says nothing about whether it is fast enough or small enough.
- **No clock domain crossing.** Everything runs on one clock. Real SoCs have dozens, and moving a signal between them safely is an entire discipline of its own.
- **No power management.** No clock gating, no power domains. Real accelerators gate the clock on idle PEs aggressively, because a PE that is not doing useful work still burns power every cycle.
- **No overflow protection.** The 32-bit accumulator cannot overflow at 4×4 with int8 (worst case 65 536), but nothing guards it if the array grows. Production designs saturate instead of wrapping, because wrapping turns a large positive number into a large negative one — a spectacular and hard-to-diagnose failure.

10 Full glossary

accumulator
A register that holds a running total, added to repeatedly. Here it is `psum_q`, 32 bits wide.
activation
Machine-learning term for an input value to a layer. In this design, an 8-bit number entering the array from the west.
active low
A signal that means "yes" when it is 0. Marked with an `_n` suffix, e.g. `rst_n`.

AGU
Address Generation Unit. The block that produces memory addresses. `agu.sv`.
always_comb
SystemVerilog block that describes combinational (memoryless) logic.
always_ff
SystemVerilog block that describes flip-flops (registers) clocked on an edge.
ASIC
Application-Specific Integrated Circuit — a chip manufactured for one purpose. Expensive to make, unbeatable once made.
back-pressure
A downstream block's ability to tell an upstream block to wait. This design has none.
combinational
Logic made only of gates, with no memory. Output depends only on the present inputs.
cycle
One tick of the clock. "Cycle *n*" here means the *n*-th rising clock edge.
de-skew
Removing a stagger so that values that were spread across several cycles line up on one.
dot product
Multiply two lists element by element and sum the products. The core operation of matrix multiply.
DUT
Device Under Test — the module a testbench is exercising.
elaboration
The stage where the tool resolves parameters, expands generate loops and connects instances. Some errors (notably port type mismatches) appear only here, not at compile time.
flip-flop
A one-bit storage element that captures its input on a clock edge. Also called a register.
FPGA
Field-Programmable Gate Array — a chip you can reconfigure. Used to prototype designs before committing to an ASIC.
FSM
Finite State Machine. A register holding "which state" plus rules for moving between states. `controller.sv`.
generate
A compile-time construct that stamps out multiple copies of hardware.
golden model
An independent computation of the expected answer, written from the specification rather than from the design.

HDL
Hardware Description Language. SystemVerilog and VHDL are the two in industrial use.
int8 / int32
Signed integers 8 and 32 bits wide. Ranges −128…127 and roughly ±2.1 billion.
latency
How long one result takes to appear, measured in cycles. Here: 7 from address to result.
localparam
A constant that cannot be overridden from outside the module. Used for internal encodings.
MAC
Multiply-ACcumulate: multiply two numbers and add the result to a running total. The fundamental operation of this chip.
Moore machine
An FSM whose outputs depend only on the current state, not on the inputs. Safer against glitches. This design's controller is one.
multiplexer (mux)
A hardware switch that selects one of several inputs. Written as `sel ? a : b`.
non-blocking assignment
`<=` — reads all right-hand sides first, then updates all left-hand sides. Required for correct register behaviour.
packed / unpacked array
Packed (`logic [31:0] x`) is one wide number you can slice. Unpacked (`logic [7:0] x [0:3]`) is four separate buses.
parameter
A compile-time constant that can be overridden at instantiation. How hardware is made reusable.
part-select
`x[base +: width]` — take `width` bits starting at `base`. Legal with a variable base, unlike `[a:b]`.
PE
Processing Element — one cell of the array. Here, one MAC plus four registers.
pipelining
Splitting work across cycles with registers between the stages, so the clock can be faster and a new result emerges every cycle.
posedge
The rising edge of the clock — the moment registers capture their inputs.
psum / partial sum
A dot product that is only partly accumulated, still travelling down a column.
quantisation
Converting a model from 32-bit floating point to small integers (usually int8) for cheaper inference.

read-before-write
Memory behaviour where a read concurrent with a write to the same address returns the *old* value.
regression
Running the full set of tests to confirm nothing broke. Here: `run_all_testbenches.ps1`.
RTL
Register Transfer Level — describing hardware as registers plus the logic that moves data between them. The abstraction all of this code is written at.
shift register
A chain of registers where each passes its value to the next every cycle. `delay_pipe` is one.
sign extension
Widening a signed number by copying its top bit. −5 in 8 bits becomes −5 in 32 bits, not 251.
simulator
Software that pretends to be the hardware so you can test it. QuestaSim here.
skew
Deliberately staggering values across cycles. The opposite of de-skew.
SoC
System on Chip — a chip containing processors, memories and accelerators together.
SRAM
Static RAM. Fast on-chip memory. All three memories here are SRAM-style.
synchronous
Everything happens on clock edges. Reset here is synchronous: it only takes effect at an edge.
synthesis
Turning RTL into actual logic gates. Not done in this project.
systolic array
A grid of simple processors through which data flows rhythmically, each cell computing and passing on. Named after the heartbeat.
testbench
Simulation-only code that drives a design and checks its outputs.
throughput
How many results emerge per cycle once the pipeline is full. Here: one result vector per cycle.
TPU
Tensor Processing Unit — Google's machine-learning accelerator. Its first version was a 256×256 weight-stationary systolic array.
weight
A learned constant in a neural network. Here, an int8 value held in a PE.

`weight-stationary`
A dataflow where weights are loaded once and stay put while data streams past them.
`wrapper`
A module whose job is to instantiate and connect other modules.
`X (unknown)`
A simulation value meaning "this wire's value is not determined". Reading uninitialised memory produces X.

11 Where to go next

To understand this design more deeply

1. Read architecture_and_dataflow.html and step through the **animated matrix-multiply figure** cycle by cycle. Everything in section 6 above becomes much more concrete once you have watched the weights shift down and the sums accumulate.
2. Open `indi_ver_rep/sim_logs/tb_accelerator_system_top.log` and read an actual simulation transcript. It prints the weight matrix, the activation vectors, and a per-transaction trace with expected values alongside.
3. Read `test_benches/tb_processing_element.sv`. It is the most approachable testbench — a single module, hand-computed expected values, and every one of its 500 random cycles traced.

To try changing something

1. **Change the flush length.** Set `FLUSH_CYCLES = ROWS` instead of `ROWS + 2` and rerun. It should still pass — the +2 is documented as belt-and-braces. Confirming that from the docs and then from the simulator is a good exercise.
2. **Break the weight order deliberately.** Make the AGU count *up* instead of down. Watch the regression fail, and note that it fails only because the weight matrix is random and non-symmetric.
3. **Add a testbench check.** Pick any existing testbench, add one more `check(...)` call, and confirm the reported count goes up by exactly one. That count invariant is what the project uses to prove no check was accidentally dropped.

To learn the field

- **SystemVerilog:** *SystemVerilog for Design* (Sutherland) and *RTL Modeling with SystemVerilog for Simulation and Synthesis* (Sutherland) are the standard references for the synthesisable subset.
- **Digital design:** *Digital Design and Computer Architecture* (Harris & Harris) — the standard first course, and it builds up to a working processor.

- **Accelerators:** the TPU paper (*In-Datacenter Performance Analysis of a Tensor Processing Unit*, Jouppi et al., 2017) is readable and describes exactly this architecture at scale. The Eyeriss papers cover the dataflow taxonomy in depth.
- **Verification:** once directed testing feels natural, the next step is constrained-random verification, functional coverage and SystemVerilog Assertions — the three things section 8 notes this project does not have.

Companion documents. architecture_and_dataflow.html — technical reference with cycle-accurate animations. · description_index.md — master file index. · design_files_description.md — per-module reference. · testbench_description.md — per-testbench reference. · verification_report.md — full results. · context_mkdwn/ — derivations and design decisions.

Design: 13 SystemVerilog modules, ~800 lines. Verification: 13 testbenches, 3 902 checks, 13/13 passing, 0 mismatches. QuestaSim 2021.1, run 2026-07-30.